

DMRC Intern Referral Portal — Handover Brief

Prepared for: DMRC IT — deployment and intranet integration
Prepared by: Shivam Sinha
Status: Pilot. Not replacing any system currently in production.
Purpose: so that our meeting starts from shared facts rather than a walkthrough.

1. What this is

A web application supporting the intern referral and onboarding process end to end:
referral → verification → offer letter → joining → clearance → completion certificate → dispatch

Part	Users	Access
Phase 1 — Referral portal	Any DMRC employee	Open to all identified staff
Phase 2 — HR dashboard	HR staff only	Three roles: HR-OPS, HR-APP, SYS-ADMIN

Offer letters and completion certificates are generated as PDF and Word files and stamped with the issuing officer's stored signature image. Every action is written to an audit ledger.

2. Technical summary

Component	Detail
Backend	Python / Django 4.2.30 with Django REST Framework
Database	MySQL 8.0.16 or newer
Frontend	Static HTML, CSS and JavaScript (Alpine.js, Bootstrap 5)
Build step	None. No Node.js, no npm, no compilation.
Application server	To be agreed — Gunicorn or uWSGI behind your web server
File storage	Server filesystem, outside the web root (\$7)
API surface	28 endpoints, every one role-protected (\$5)

3. The integration

This portal authenticates nobody. No login screen, no stored passwords. Identity comes from DMRC's existing employee login; the portal only decides what an already-identified employee may do here.

DMRC employee login	→ WHO the person is	(your system)
This portal	→ WHAT they may do here	(this system)

The entire integration is one Python method, in `portal/identity/intranet.py`. It returns the employee code of the signed-in user, matching `employees.employee_code`, or nothing for an unauthenticated request — in which case the portal replies 401.

`portal/identity/base.py` documents the contract in full. `intranet.py` sketches four common patterns: reverse-proxy header, SAML/OIDC, shared session, LDAP/Active Directory.

Request: please nominate someone from the payslip login team to read those two files. Nothing else in the codebase needs to change.

Once a code is resolved, two tables decide everything:

- `employees` — any employee found here may use the Phase 1 referral portal.
- `users` — dashboard accounts only. An employee with no row here cannot reach the HR dashboard at all. This is the default for all staff and is intentional. Roles are granted deliberately by a SYS-ADMIN, never inherited from the intranet.

4. Questions

Five I'd like answered in the meeting

#	Question	Why it blocks me
1	What does a real DMRC employee code look like? Three examples, please — and is the format consistent across all staff?	Everything joins on this. If your login returns <code>40255</code> while the directory export loaded <code>E40255</code> , nothing matches and no employee can use the portal at all.
2	Will the pages and the API be served from the same address , with your web server forwarding <code>/api/</code> to the application?	Determines how I fix the 61 hardcoded addresses in §8. Same address lets me use relative links and removes browser-permission configuration entirely.
3	Does your MySQL require an encrypted connection? If so, where is the CA certificate file on that server?	Determines whether the application can connect at all. Now configurable — I just need the values.
4	Can you export the employee directory? A CSV with employee code, name, designation, department and official email.	The portal cannot be used by anybody until this table is populated. See §6.
5	Who should hold the first SYS-ADMIN account? Name, employee code, email.	Nobody can administer the system until this row exists, and it cannot be created through the interface. See §6.

Twelve that can be answered in writing

#	Question
6	Which Linux distribution, and what does <code>python3 --version</code> print on the target server?
7	What hostname will the portal be reached at? (Needed for <code>ALLOWED_HOSTS</code> .)
8	Which MySQL version? One validation rule is enforced from 8.0.16 onward and silently ignored before it.
9	How does the payslip login identify a user to a downstream application, and who can I talk to about it?
10	Is the portal to be reachable only from within the DMRC network?
11	Who owns and maintains this application after handover?
12	From the browser on an HR user's PC — not from the server — is the public internet reachable? Specifically <code>cdn.jsdelivr.net</code> and <code>fonts.googleapis.com</code> . See §8, gap 2.
13	What browsers do HR staff use, and which versions? The interface requires a modern browser.
14	Mail relay for the notification system: hostname and port; does it need credentials; what "From" address may the portal use; and can it deliver to recipients outside DMRC? Certificate dispatch goes to candidates, who are students on personal email addresses.
15	How should employee records be kept current — new joiners, leavers, transfers, promotions? A periodic re-export is sufficient for a pilot.
16	Does any other DMRC system need to call this portal's API? If so, that needs a service account or token — the current model assumes a person in a browser.
17	Does DMRC have a naming convention for databases, or is <code>dmrc_internship_portal</code> acceptable?
18	Where should application logs be written? Is there a central location?

5. Security posture

- **Authorisation is decided server-side on every request**, read from the `users` table. Nothing the browser sends is trusted. All 28 endpoints are guarded; there is no unprotected endpoint. Six accept any known employee (the referral portal); the other twenty-two require a specific dashboard role.
- Issuing offer letters and certificates is restricted to HR-APP. Account management, cycle configuration and archives are restricted to SYS-ADMIN. Clearance and handover are HR-OPS and SYS-ADMIN.
- **Uploaded candidate documents** are stored outside every directory the web server serves. One authenticated endpoint reaches them: it checks the caller's role, expires the link after ten minutes, streams the file, and writes every access to the audit ledger.
- **Generated letters and certificates** are stored the same way, for the same reason.
- **Signature images** are stored more strictly again: served only to the officer they belong to and to a SYS-ADMIN.
- **Django's built-in admin is deliberately not routed.** It would be a second, parallel username-and-password login unrelated to the employee directory. Nothing in the project uses it.
- **Two production safety checks** refuse to start the server if the secret key is still the development default, or if the development identity provider is still configured.
- Errors and warnings are written to a rotating log file (§7).

Three items for your security team

1. **Django 4.2 reached end of extended support on 7 April 2026** and receives no further security patches. Django 5.2 LTS is supported until April 2028. I would rather complete this upgrade before go-live than after; it needs the server's Python version (Q6) to scope.
2. **If integration uses a reverse-proxy header** (Pattern A), two conditions are mandatory: the proxy must strip that header from inbound client requests, **and** the application must not be reachable directly, bypassing the proxy. Otherwise any user on the network can set the header themselves and impersonate any employee, including a SYS-ADMIN. Bind the application to localhost, or firewall its port to the proxy.
3. **Frontend libraries load from public CDNs at unpinned versions** — Alpine.js as `3.x.x`, Flatpickr with no version at all. The application's behaviour can change without any change to my code. Self-hosting fixed copies resolves this and gap 2 together, and I intend to do it regardless of the answer to Q12.

Two things you should know rather than discover

- The data export endpoint is available to **all three dashboard roles**, including the least privileged. If exports should be restricted to administrators, say so and it is a one-line change.
- The audit ledger is an ordinary database table. It is protected by database access control, not by design — it is not tamper-proof against someone with direct database access.

6. Deployment — the two steps that are easy to miss

`DB/Intern_Portal.sql` is the authoritative schema. It creates the database, all 27 tables, and seeds reference data.

⚠ **Run `Intern_Portal.sql` exactly once, against an empty database.**

It begins by dropping every table it creates. Re-running it against a database holding live data will delete that data permanently.

⚠ **Do not run anything in `DB/history/` or `DB/dev_only/`.**

`history/` holds the development migration record. Every change it makes is already in the master script, and one file uses TiDB-specific syntax that fails on MySQL outright. `dev_only/` holds reset and seeding utilities — one inserts fictitious employees. Both folders carry a README explaining this.

Django is configured never to alter the schema — all models are declared unmanaged.

⚠ Run Django's migrations with `--fake` on a new installation.

```
python manage.py migrate portal --fake
python manage.py migrate
```

The portal's three migrations issue raw SQL to upgrade a database built *before* those changes existed. `Intern_Portal.sql` already contains their result, so running them normally against a fresh database fails — migration 0002 adds columns the master script already has, and 0003 drops columns it never created. The first command records them as applied without executing them; the second then creates only Django's own internal tables. Order matters. On an existing database that predates these changes, run them normally.

The schema also ships five analytical views (`vw_hr_application_status_tracker` and four others). The portal itself never reads them; they exist so that a reporting tool or Excel can be pointed at the database directly.

Then — the portal will not work until both of these are done

A. Populate `employees`. This portal does not own employee identity: it reads that table and never writes to it. On an empty table every request is refused with a 401, including the Phase 1 referral portal. The employee code must be *exactly* the string your login system returns.

```
INSERT INTO employees (employee_code, full_name, designation, department_id, official_email)
VALUES ('EMP-1001', 'NAME AS IN DIRECTORY', 'DESIGNATION AS IN DIRECTORY',
       (SELECT department_id FROM departments WHERE department_name = 'IT'),
       'person@dmrc.org');
```

The designation is printed beneath the signature on every offer letter that person signs, so it must be kept current — see Q15.

B. Create the first SYS-ADMIN. Dashboard accounts are created from the IAM screen, which requires the caller to already hold the SYS-ADMIN role. The first account therefore cannot be created through the interface:

```
INSERT INTO users (role_id, employee_id, username, email)
VALUES ((SELECT role_id FROM roles WHERE role_name = 'SYS-ADMIN'),
       (SELECT employee_id FROM employees WHERE employee_code = 'EMP-1001'),
       'chosen.username', 'person@dmrc.org');
```

That person can then provision every other account from the dashboard.

Two operational notes

- **Never start this with `manage.py runserver` on a server.** That turns on development mode, which disables both safety checks and re-enables the identity provider that trusts a request header. Setting `DEBUG=False` in `.env` — which the example file now marks as required — makes that command fail loudly instead.
- **Restrict `.env`** so that only the account running the application can read it. It holds the database password in plain text.

7. File storage, backup and logging

Uploaded documents, generated letters and signature images are stored on the **server filesystem**, outside the web root — not in the database:

Back End/protected_documents/	candidate uploads
Back End/generated_documents/	offer letters and certificates
Back End/signatures/	officer signature images
Back End/quarantine/	superseded and rejected files

Created automatically on first use. They require:

1. **Persistent storage** surviving redeployment.
2. **Backup together with the database.** A database-only backup restores records pointing at files that no longer exist, and closing an archive cycle fails when a referenced file is missing.
3. **Exclusion from any temporary-file cleanup policy.**

A restore has never been tested. I would suggest testing one during the pilot rather than after it.

Logging: warnings and errors are written to `Back End/logs/portal.log`, rotating at 5 MB with five copies retained. `LOG_DIR` in `.env` redirects this to a central location (Q18).

8. Known gaps — stated plainly

#	Gap	Status
1	Database connection may need TLS settings this server does not use. Previously hardcoded to a macOS certificate path with no way to change it.	Now configurable via <code>DB_SSL</code> and <code>DB_SSL_CA</code> in <code>.env</code> . Needs the values from Q3.
2	Frontend libraries load from public CDNs. Tested with the internet disconnected: Phase-1 rendered the header and nothing else — the referral wizard did not exist on the page. The HR dashboard rendered an unstyled skeleton with no data. Seven resources failed on Phase-1 and six on the dashboard, including Alpine.js, which builds every screen in both portals.	Mine to fix, roughly one hour. Q12 decides urgency, not whether it is done.
3	61 hardcoded developer addresses (<code>http://127.0.0.1:8000</code>) — 47 in the dashboard script, 13 in the referral script, 1 in the dashboard HTML. Every one fails on the intranet.	Mine to fix, blocked on Q2. About one day including testing every screen.
4	Automated email notifications. Certificate dispatch is currently recorded as PENDING and the notifications table is never written to.	In development. My manager has approved delivering this after integration, so that DMRC IT can begin testing the integration now rather than waiting. Blocked on Q14.
5	Django 4.2 out of security support (\$5).	Mine to fix, blocked on Q6.
6	No upload size limit on candidate documents. The database column exists and is unused. File <i>types</i> are validated per document type.	Small fix, needs a limit from HR.
7	No automated test suite. Tested by hand against the full workflow.	Would be added before this handles anything beyond a pilot.
8	No way to deactivate a departed employee. The <code>employees</code> table has no active flag, and the database refuses to delete anyone who has ever referred a candidate — correctly, since that would destroy referral history. Access closes upstream when their intranet login is disabled.	Small schema change, cheaper to make before the pilot fills with data. Dashboard <i>accounts</i> can already be deactivated.
9	No API versioning or rate limiting.	Acceptable for a single internal consumer; worth adding if Q16 is yes.
10	Ownership after handover undefined (Q11).	For discussion.

9. Personal data

Candidate records include **Aadhaar numbers, stored in full and unencrypted**, in the `students` and `archived_applications` tables. The document catalogue flags Aadhaar as requiring the candidate's consent; the consent checkbox and the number field both follow that flag, and consent is captured at upload. Every document access is written to the audit ledger.

Whether plain-text storage meets DMRC's obligations — under the DPDP Act 2023 and the DPDP Rules 2025, and under DMRC's own policy as a State instrumentality — is a decision for DMRC, not for me. **If encryption, masking, or a shorter retention period is required, tell me the requirement and I will implement it.** The same policy should be applied to the file storage directories in §7, which hold Aadhaar document images.

10. What I propose for the meeting

1. Answer the five questions in §4. Most take seconds.
2. Identify the person who will look at the identity contract.
3. Agree who does what, and in what order.
4. Agree what "working pilot" means, so we both know when we are finished.

Prepared ahead of our meeting so that time is spent on decisions rather than description.